

AI-Powered SOC Platform — Comprehensive Documentation

Project Overview

The **AI-Powered SOC Platform** is a modular, production-ready security operations center (SOC) system for real-time network threat detection, automated incident response, and security workflow management. It uses a Random Forest classifier trained on the CIC-IDS2017 dataset (78 network flow features, reduced via IncrementalPCA) to detect multi-class network attacks.

Key capabilities:

- Real-time packet capture (Scapy or PyShark) with per-flow feature extraction
- ML-based threat classification with sub-millisecond inference
- WebSocket-based live alert streaming to the dashboard
- Automated response engine with 11 actions (firewall block, rate-limit, quarantine, etc.)
- Full incident management lifecycle (create → assign → investigate → close)
- Role-based access control (admin/analyst/viewer) with JWT authentication
- Threat intelligence aggregation (IP reputation, IOC management)
- Report generation (daily, weekly, monthly, incident, executive) in PDF/CSV/JSON
- Analytics, trend analysis, and threat forecasting
- Notification routing (email, Telegram, webhook) per severity
- Prometheus/Grafana monitoring and ELK stack logging
- Docker Compose deployment with horizontal scaling support

Project root: C:\Users\Mukhil\Downloads\Project\ids-platform-milestone3

Version: Milestone 3 (Python 3.14, React 19, FastAPI)

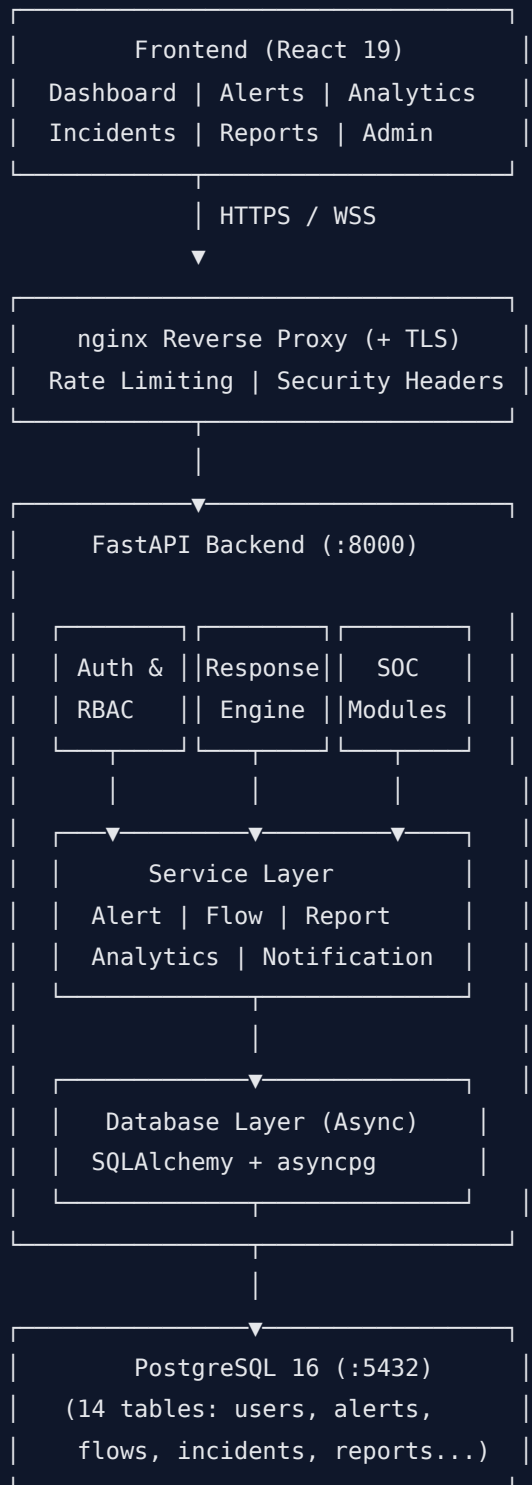
Technology Stack

Layer	Technology	Version
Backend framework	FastAPI (Python)	3.14 runtime
API docs	Swagger UI + ReDoc	auto
Database	PostgreSQL	16 (Alpine)
ORM	SQLAlchemy (async)	2.x
DB driver	asyncpg	latest
ML model	scikit-learn Random Forest	via joblib
ML features	IncrementalPCA (dimensionality reduction)	scikit-learn
Data processing	pandas, numpy	latest

Packet capture	Scapy / PyShark	latest
Auth	JWT (HS256) via PyJWT	custom
Password hashing	bcrypt via passlib	latest
Frontend framework	React	19.2.7
Frontend language	TypeScript	6.0
Build tool	Vite	8.1
Styling	Tailwind CSS	4.3
Server state	TanStack React Query	5.101
HTTP client	Axios	1.18
Charts	Recharts	3.10
Routing	React Router DOM	7.18
Linting	oxlint	1.71
Reverse proxy	nginx	1.27
Monitoring	Prometheus + Grafana	latest
Logging	ELK Stack (Elasticsearch, Logstash, Kibana, Filebeat)	8.12
Orchestration	Docker Compose	v3.9
Testing	pytest + pytest-asyncio	latest

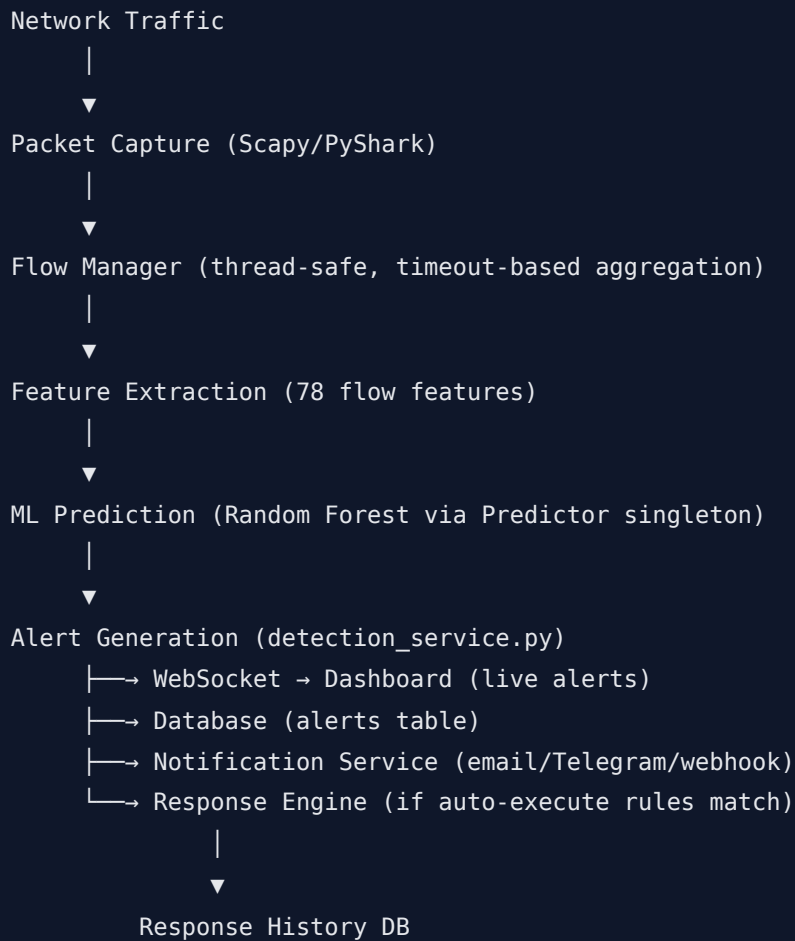
Architecture

System Diagram

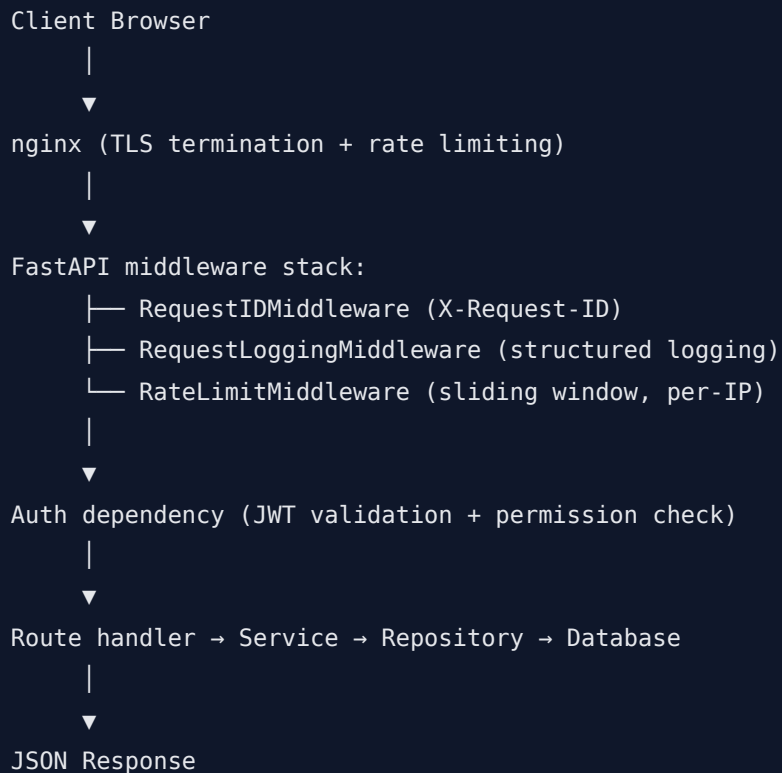


Data Flow

Real-time Detection Pipeline



API Request Flow



Folder Structure

```

ids-platform-milestone3/
├─ backend/
|   ├─ main.py                # FastAPI app entry point
|   └─ core/
|       ├─ config.py         # pydantic-settings configuration
|       ├─ logging.py        # Structured logging utilities
|       └─ middleware.py     # RequestID, RequestLogging, RateLimit
|   └─ api/
|       ├─ router.py         # Central router aggregation
|       └─ routes/
|           ├─ auth.py       # Authentication & user management
|           ├─ alerts.py     # Alert CRUD
|           ├─ flows.py      # Network flow listing
|           ├─ statistics.py # Threat statistics
|           ├─ health.py     # Health checks
|           ├─ predictions.py # ML prediction endpoint
|           ├─ detection.py  # Detection control
|           ├─ capture.py    # Packet capture control
|           ├─ model.py      # Model management
|           ├─ incidents.py  # Incident management
|           ├─ responses.py  # Response engine
|           ├─ reports.py    # Report generation
|           ├─ analytics.py  # Analytics, trends, forecast
|           ├─ threat_intel.py # Threat intelligence
|           ├─ notifications.py # Notification rules
|           ├─ testing.py    # Attack simulation
|           └─ system.py     # System health
|   └─ database/
|       ├─ connection.py    # Async engine + session factory
|       ├─ models.py        # SQLAlchemy ORM models
|       ├─ migrations/     # (Empty) Alembic directory
|       └─ repositories/
|           ├─ alerts.py    # Alert DB operations
|           ├─ flows.py     # Flow DB operations
|           ├─ packets.py   # Packet DB operations
|           └─ statistics.py # Statistics queries
|   └─ auth/
|       ├─ models.py        # Auth data models (Pydantic)
|       ├─ routes.py        # Auth REST handlers
|       ├─ authentication.py # JWT token creation/validation
|       ├─ authorization.py # Permission checking
|       ├─ jwt_manager.py   # Token lifecycle management
|       ├─ permissions.py   # Role-permission mapping
|       ├─ password.py      # Password hashing/verification
|       ├─ dependencies.py  # FastAPI dependency injection
|       ├─ repositories.py  # User/session/audit DB ops
|       ├─ audit.py         # Audit logging
|       └─ seed.py          # Default admin user seeding

```

```

| | └─ ml/
| |   │ └─ predictor.py           # Model singleton (load + predict)
| |   │ └─ inference.py          # Inference pipeline
| |   │ └─ schemas.py            # Prediction I/O schemas
| |   │ └─ artifacts.py          # Model save/load/versioning
| |   │ └─ feature_engineering.py # Scaling + PCA + transform
| |   │ └─ preprocessing.py      # Raw data preprocessing
| |   │ └─ constants.py          # Feature column names
| |   │ └─ confidence.py         # Confidence score calculation
| |   │ └─ severity.py           # Severity classification
| |   │ └─ training.py           # Model training utilities
| |   │ └─ evaluation.py         # Model evaluation metrics
| |   │ └─ validator.py          # Input validation
| |   │ └─ train_all_models.py    # Batch training script
| | └─ detection/
| |   │ └─ detection_service.py   # Orchestrates capture→predict→alert
| └─ packet_capture/
|   │ └─ capture.py              # Capture manager (Scapy)
|   │ └─ flow_manager.py         # Thread-safe flow aggregation
|   │ └─ packet_parser.py        # Raw packet parsing
|   │ └─ flow_features.py        # Flow-level feature computation
|   │ └─ feature_mapper.py       # Map features to model columns
|   │ └─ interface.py            # Network interface listing
|   │ └─ packet_filter.py        # BPF filter support
|   │ └─ tshark_capture.py       # PyShark-based capture
| └─ response_engine/
|   │ └─ response_service.py     # Response orchestration
|   │ └─ executor.py            # Action execution engine
|   │ └─ rules.py               # Severity-based rule evaluation
|   │ └─ registry.py            # Action registration
|   │ └─ registrations.py       # Built-in action registrations
|   │ └─ recovery.py            # Rollback support
|   │ └─ simulation.py          # Simulation mode
|   │ └─ firewall.py            # Firewall rule management
|   │ └─ quarantine.py          # Host isolation
|   │ └─ notifications.py       # Response notifications
| └─ incidents/
|   │ └─ incident_service.py     # Incident management service
| └─ websocket/
|   │ └─ ws_manager.py          # WebSocket connection manager
| └─ notifications/
|   │ └─ notification_manager.py # Notification dispatch
|   │ └─ email_service.py        # SMTP email
|   │ └─ telegram_service.py     # Telegram bot
|   │ └─ webhook_service.py      # HTTP webhook
| └─ reports/
|   │ └─ report_service.py       # Report generation orchestration
|   │ └─ pdf_generator.py        # PDF report rendering

```

```

| | | └─ csv_export.py          # CSV export
| | |   └─ templates/          # Report templates
| | └─ threat_intelligence/
| | |   └─ reputation.py        # IP reputation checking
| | |   └─ indicators.py        # IOC management
| | |   └─ enrichment.py        # Threat enrichment
| | |   └─ feeds.py             # External feed integration
| | └─ analytics/
| | |   └─ analytics_service.py # Analytics aggregation
| | |   └─ trends.py            # Trend computation
| | |   └─ forecasting.py       # Threat forecasting
| | └─ services/
| | |   └─ prediction_service.py # Prediction business logic
| | |   └─ alert_service.py      # Alert business logic
| | |   └─ history_service.py    # Flow history
| | |   └─ capture_service.py    # Capture orchestration
| | |   └─ statistics_service.py # Statistics computation
| | |   └─ testing_service.py    # Attack simulation
| | └─ utils/
| | |   └─ logger.py            # Python logging config
| └─ scripts/
|   └─ train_pipeline.py        # End-to-end training CLI
└─ frontend/
  └─ src/
    └─ main.tsx                # React entry point
    └─ App.tsx                 # Router + providers
    └─ index.css               # Tailwind imports
    └─ App.css                 # Global styles
    └─ pages/                  # 22 page components
    └─ components/             # Reusable UI components
    |   └─ layout/              # Layout, Sidebar, TopNav
    |   └─ common/              # ErrorDisplay, LoadingSkeleton, SeverityBadge
    |   └─ dashboard/           # StatCard
    |   └─ charts/              # BarChart, PieChart, SparkLine
    |   └─ alerts/              # (empty)
    |   └─ tables/              # (empty)
    |   └─ statistics/          # (empty)
    └─ services/               # 14 API service modules
    └─ auth/                   # AuthContext, authService, ProtectedRoute
    └─ context/                # ThemeContext, AlertContext, PacketStreamContext
    └─ hooks/                  # useWebSocket, useMetricsHistory
    └─ types/                  # TypeScript type definitions
    └─ utils/                  # constants, formatters, highlight
  └─ package.json
└─ tests/                      # 25 test files
└─ deployment/                 # Docker, nginx, monitoring, logging
└─ docs/                       # 11 documentation files
└─ examples/                   # Sample prediction input

```

```
└─ models/                # Versioned ML model artifacts (v1-v10)
└─ .env                   # Environment configuration
└─ requirements.txt        # Python dependencies
└─ pytest.ini             # Pytest configuration
└─ conftest.py            # Root conftest
└─ CHangelog.md            # Version history
└─ README.md              # Project readme
```

Feature Documentation

1. Packet Capture (backend/packet_capture/)

Purpose: Capture live network traffic from a network interface and extract per-flow features for ML inference.

Files involved:

File	Responsibility
capture.py	CaptureManager — Async capture loop using Scapy sniff() , runs in a background thread, pushes raw packets to FlowManager
tshark_capture.py	TSharkCaptureManager — Alternative capture backend using PyShark; provides live_capture() generator
flow_manager.py	FlowManager — Thread-safe dictionary of active flows keyed by (src_ip, dst_ip, src_port, dst_port, protocol) ; flows timeout after configurable inactivity period (default 120s)
packet_parser.py	PacketParser — Extracts IP, TCP, UDP, ICMP fields from raw Scapy packets
flow_features.py	Computes 78 flow-level features (durations, packet counts, byte counts, mean/std/max/min of packet lengths, IAT, flags, window sizes, etc.)
feature_mapper.py	Maps computed flow features to the canonical 78-column feature vector expected by the ML model
interface.py	Lists available network interfaces with metadata (name, description, index, availability)
packet_filter.py	Applies BPF (Berkeley Packet Filter) expressions to filter captured traffic

Key implementation details:

- CaptureManager uses a background thread with asyncio queue to decouple capture from processing
- FlowManager uses threading.Lock for thread-safe flow updates; flows are finalized and published via callback when they timeout
- Flow timeout is configurable via capture.py 's FlowManager init params
- TSharkCaptureManager requires tshark installed and available in PATH
- The capture service (services/capture_service.py) orchestrates start/stop/status

2. ML Pipeline (backend/ml/)

Purpose: Train, version, load, and run inference with a multi-class Random Forest classifier for network attack detection.

Training pipeline (`scripts/train_pipeline.py`):

1. Load dataset (CIC-IDS2017 CSV)
2. Preprocess: handle missing values, encode labels, split features/target
3. Feature engineering: `StandardScaler` → `IncrementalPCA` (reduce to `n_components` features)
4. Train Random Forest model with optional hyperparameter grid search
5. Evaluate on held-out test set (accuracy, precision, recall, F1)
6. Save artifact bundle: model (joblib), scaler (joblib), PCA (joblib), feature names, metadata JSON

Inference pipeline (`inference.py`):

1. Receive 78 raw feature values
2. Scale via fitted `StandardScaler`
3. Transform via fitted `IncrementalPCA`
4. Predict class probabilities via Random Forest
5. Determine attack type, confidence, severity

Key classes:

Class/Module	Role
<code>Predictor</code> (<code>predictor.py</code>)	Singleton; lazy-loads model on first <code>predict()</code> call; thread-safe with <code>asyncio.Lock</code> ; caches loaded model in memory
<code>FeatureEngineer</code> (<code>feature_engineering.py</code>)	Fits or loads scaler + PCA; <code>fit()</code> and <code>transform_to_frame()</code> methods
<code>artifacts.py</code>	<code>save_model_bundle()</code> , <code>load_model_bundle()</code> , <code>next_version_dir()</code> , <code>build_metadata()</code> — manages versioned model artifacts on disk
<code>train_all_models.py</code>	Batch training over multiple parameter sets; saves each as versioned artifact
<code>confidence.py</code>	Calculates confidence score from prediction probabilities
<code>severity.py</code>	Maps attack type + confidence to severity level (Critical/High/Medium/Low)

Model persistence:

- Artifacts stored in `models/v{1..10}/` directories
- Each version directory contains: `model.joblib` , `scaler.joblib` , `pca.joblib` , `feature_names.json` , `metadata.json`
- Models are never overwritten in place — each training run creates a new version directory
- Active model version is tracked in memory (not persisted in DB)

3. Detection Service (`backend/detection/detection_service.py`)

Purpose: Orchestrates the full detection pipeline: capture → flow → predict → alert.

Flow:

1. Starts packet capture on a specified interface
2. Receives parsed packets from `CaptureManager`
3. Feeds packets into `FlowManager` for flow aggregation
4. When a flow times out, computes 78 flow features
5. Calls `Predictor.predict()` with feature vector
6. If threat detected (confidence > threshold), creates an `Alert` record
7. Broadcasts alert via WebSocket
8. Evaluates response rules for auto-execution
9. Sends notifications based on notification rules

4. Auth & Authorization (`backend/auth/`)

Purpose: User authentication, session management, role-based access control, and audit logging.

Authentication flow:

1. `POST /auth/login` → validate credentials → issue `access_token` (JWT, 60min) + `refresh_token` (JWT, 7 days)
2. `POST /auth/refresh` → validate refresh token → issue new token pair
3. `POST /auth/logout` → revoke refresh token (mark as revoked in `user_sessions` table)
4. Protected endpoints use `get_current_user` dependency that validates Bearer JWT

Key files:

File	Role
<code>authentication.py</code>	<code>create_access_token()</code> , <code>create_refresh_token()</code> , <code>decode_token()</code> using PyJWT (HS256)
<code>jwt_manager.py</code>	Token pair generation, refresh, revocation, cleanup of expired sessions
<code>authorization.py</code>	<code>require_permission()</code> dependency factory; checks user role against permission map
<code>permissions.py</code>	Defines 3 roles with 15 granular permissions; permission hierarchy (admin inherits all)
<code>password.py</code>	<code>hash_password()</code> , <code>verify_password()</code> using bcrypt via passlib
<code>repositories.py</code>	User CRUD, session management, audit log queries
<code>dependencies.py</code>	FastAPI Depends callables: <code>get_current_user</code> , <code>get_optional_user</code> , <code>require_permission</code>
<code>audit.py</code>	<code>log_auth_event()</code> — records auth actions (login, logout, permission denied, etc.) to <code>auth_audit_logs</code>
<code>seed.py</code>	Creates default admin user (<code>admin</code> / <code>admin123</code>) on first startup if no users exist

Roles and permissions:

Role	Permissions
admin	All 15 permissions (manage:users, manage:policies, view:audit, execute:response, execute:rollback, view:alerts, view:statistics, view:responses, view:policies, manage:notifications)

analyst	view:alerts, view:statistics, view:responses, view:policies, execute:response, execute:rollback
viewer	view:alerts, view:statistics, view:responses, view:policies

5. Response Engine (backend/response_engine/)

Purpose: Automated or manual execution of response actions against detected threats.

Key files:

File	Role
response_service.py	Orchestrates execution: validates rules, creates response record, dispatches to executor
executor.py	ResponseExecutor — executes action chains sequentially; records results per action
rules.py	ResponseRules — loads default_rules.json ; matches alert severity + attack type to configured actions
registry.py	ResponseRegistry — singleton mapping action names → callable handlers
registrations.py	Registers all 11 built-in actions at startup
simulation.py	Simulation mode wrapper — logs intended action without executing
recovery.py	Rollback support — stores undo state for reversible actions (firewall unblock, rate-limit remove)
firewall.py	Platform-specific firewall rule management (iptables/nftables simulation)
quarantine.py	Host isolation via ARP spoofing or network namespace manipulation (simulation-only)

11 registered actions:

Action	Simulation only?	Reversible?	Description
block_ip	No	Yes	Blocks IP via firewall rule
rate_limit	No	Yes	Rate-limits traffic from source IP
kill_process	Yes	No	Kills malicious process on host
isolate_host	Yes	No	Network-isolates compromised host
quarantine_host	Yes	No	Quarantines host via VLAN/ACL
notify_admin	No	N/A	Sends admin notification
log_incident	No	N/A	Creates incident record
update_firewall	No	Yes	Updates firewall rule set
revoke_session	Yes	No	Revokes user session
restart_service	Yes	No	Restarts affected service
vpn_isolation	Yes	Yes	Isolates via VPN routing

Execution modes:

- `simulation` : Logs what would happen; no real system changes
- `live` : Executes actions with real system impact
- `auto` : Automatically triggered by response rules (can be simulation or live based on rule config)

Response rules (loaded from `default_rules.json`):

- Maps severity × attack_type combinations to action lists with `auto_execute` and `simulation_mode` flags
- Rules are editable via API (`PUT /responses/rules`)

6. WebSocket System (`backend/websocket/ws_manager.py`)

Purpose: Manages WebSocket connections for real-time data streaming.

Key features:

- `ConnectionManager` class with `connect()` / `disconnect()` / `broadcast()` / `send_personal()`
- Tracks active connections per channel (alerts, packets, statistics)
- Heartbeat via periodic ping/pong (every 15s)
- Automatic cleanup of stale connections
- Three event types: `alert`, `packet`, `statistics`
- Broadcasting uses `asyncio.gather()` for concurrent delivery

Endpoints:

- `WS /ws/alerts` — Live alert stream
- `WS /ws/packets` — Live packet stream (requires active capture)

Frontend consumers:

- `websocketService.ts` (singleton `wsService`) — Class-based WebSocket client with auto-reconnect (exponential backoff 3-15s), event subscription, heartbeat
- `AlertContext.tsx` — Subscribes to alert events, maintains rolling buffer (max 200 alerts)
- `PacketStreamContext.tsx` — Subscribes to packet events, maintains rolling buffer (max 200 packets)

7. Notification System (`backend/notifications/`)

Purpose: Route alerts to external notification channels based on severity-based rules.

Channels provided:

Channel	File	Status
Email (SMTP)	<code>email_service.py</code>	Logs only — SMTP credentials in <code>.env</code> but no actual SMTP client integration
Telegram	<code>telegram_service.py</code>	Logs only — bot token and chat ID in <code>.env</code> but no actual API calls
Webhook	<code>webhook_service.py</code>	Logs only — no HTTP client integration

Notification rules:

- Stored in `notification_rules` table
- Map severity levels to channel lists (`{"Critical": ["email", "telegram"], "High": ["email"]}`)

- Editable via API (GET/PUT /notifications/rules)

Important note: All notification services currently log messages to console only. No real SMTP, Telegram API, or HTTP webhook client has been integrated. This is simulation/skeleton code.

8. Threat Intelligence (backend/threat_intelligence/)

Purpose: IP reputation checking, IOC (Indicator of Compromise) management, and external feed integration.

Key files:

File	Role
reputation.py	check_ip_reputation() — queries configured providers, returns reputation score
indicators.py	CRUD for threat_indicator_cache table — stores IOCs
enrichment.py	Enriches alerts with threat intelligence data
feeds.py	Simulates fetching from external feeds (AlienVault OTX, VirusTotal)

Important note: All threat intelligence feeds return empty lists. No real API keys are configured in .env (AlienVault OTX and VirusTotal keys are empty strings). The reputation module returns "unknown" for all IPs. This is simulation infrastructure.

9. Incident Management (backend/incidents/)

Purpose: Full incident lifecycle management.

Features:

- Create incidents from alerts (associate multiple alerts)
- Assign incidents to analysts
- Add notes and timeline entries
- Track status (open → investigating → resolved → closed)
- Severity and priority classification

Tables involved: incidents , incident_notes , incident_timeline

10. Reports (backend/reports/)

Purpose: Generate security reports in multiple formats.

Report types: daily, weekly, monthly, incident-specific, executive summary

Output formats: PDF (via pdf_generator.py), CSV (via csv_export.py), JSON

Key files:

File	Role
report_service.py	Orchestrates report generation: queries data, calls formatter, saves file, creates DB record
pdf_generator.py	Generates PDF reports with tables and charts (uses ReportLab or similar)
csv_export.py	Exports tabular data to CSV format

11. Analytics (backend/analytics/)

Purpose: Historical analysis, trend detection, and threat forecasting.

Key files:

File	Role
analytics_service.py	Aggregates alert data into analytics summaries (top attack types, severity distribution, top IPs, timeline, response time stats)
trends.py	Computes daily attack counts and type/severity distribution over configurable period (default 30 days)
forecasting.py	Uses simple statistical method (moving average / linear extrapolation) to forecast future threat counts

12. Attack Simulation (backend/services/testing_service.py)

Purpose: Generate synthetic test events to validate the detection and response pipeline.

Simulated attack types: Port Scan, SYN Flood, UDP Flood, ICMP Flood, SSH Brute Force, DNS Attack, Custom Test Event

Flow:

1. POST /testing/simulate?attack_type=port_scan
2. Creates a synthetic alert with realistic feature values
3. Injects alert into the normal detection pipeline (creates DB record, broadcasts via WebSocket, evaluates response rules)
4. Returns the generated alert details

API Documentation

Authentication Endpoints

Method	Path	Auth	Description
POST	/auth/register	No	Register new user
POST	/auth/login	No	Login, returns access + refresh tokens
POST	/auth/refresh	No	Refresh access token
POST	/auth/logout	Yes	Revoke refresh token
GET	/auth/me	Yes	Get current user profile
PUT	/auth/me	Yes	Update profile (full_name, email)
PUT	/auth/change-password	Yes	Change password
GET	/auth/permissions	Yes	Get current user's role + permissions

GET	/auth/users	admin	List all users
PUT	/auth/users/{id}	admin	Update user (role, is_active)
GET	/auth/audit	admin	Get audit logs
DELETE	/auth/audit	admin	Clear audit logs

Threat Detection Endpoints

Method	Path	Auth	Description
GET	/health	No	Health check (DB, model, uptime)
GET	/model/info	No	Model metadata
POST	/predict	No	Run ML prediction on feature vector
GET	/alerts	Yes	List alerts (paginated, filterable)
GET	/alerts/latest	Yes	Latest N alerts
GET	/alerts/{id}	Yes	Single alert details
GET	/flows	Yes	List network flows (paginated, filterable)
GET	/statistics	Yes	Threat statistics summary
GET	/system/health	Yes	System health (CPU, memory, disk, network)
GET	/attacks/top	Yes	Top attack types
WS	/ws/alerts	No	Real-time alert stream
WS	/ws/packets	No	Real-time packet stream

Model Management Endpoints

Method	Path	Auth	Description
GET	/model/info	No	Active model metadata
GET	/model/versions	Yes	List all available model versions
POST	/model/switch/{version}	Yes	Switch active model version
GET	/model/all	Yes	All models with full metadata
GET	/model/active-code	Yes	Active model's pipeline + training code
GET	/model/code/{version}	Yes	Specific version's pipeline + training code

Capture Control Endpoints

Method	Path	Auth	Description
--------	------	------	-------------

GET	/capture/interfaces	Yes	List network interfaces
POST	/capture/start	Yes	Start capture on interface
POST	/capture/stop	Yes	Stop active capture
GET	/capture/status	Yes	Capture status + stats

Response Engine Endpoints

Method	Path	Permission	Description
POST	/responses/execute	execute:response	Execute response action(s)
POST	/responses/rollback	execute:rollback	Rollback previous response
GET	/responses	view:responses	List response history
GET	/responses/{id}	view:responses	Response detail
GET	/responses/export	view:responses	Export history (JSON/CSV)
GET	/responses/rules	view:policies	Get auto-response rules
PUT	/responses/rules	manage:policies	Update auto-response rules
GET	/responses/actions/list	view:responses	List available actions

Incident Endpoints

Method	Path	Permission	Description
POST	/incidents	view:alerts	Create incident
GET	/incidents	view:alerts	List incidents
GET	/incidents/{id}	view:alerts	Incident details
PUT	/incidents/{id}	view:alerts	Update incident
POST	/incidents/{id}/notes	view:alerts	Add note

Report Endpoints

Method	Path	Permission	Description
POST	/reports/generate	view:alerts	Generate report
GET	/reports	view:alerts	List reports
GET	/reports/{id}	view:alerts	Report details
GET	/reports/{id}/download	view:alerts	Download report file
DELETE	/reports/{id}	view:alerts	Delete report

POST	/reports/{id}/regenerate	view:alerts	Regenerate report
------	--------------------------	-------------	-------------------

Analytics Endpoints

Method	Path	Permission	Description
GET	/analytics	view:statistics	Analytics overview
GET	/analytics/trends	view:statistics	Threat trends (default 30 days)
GET	/analytics/forecast	view:statistics	Threat forecast (default 7 days)

Threat Intelligence Endpoints

Method	Path	Permission	Description
GET	/threat-intelligence	view:alerts	Threat intel status summary
GET	/threat-intelligence/reputation/{ip}	view:alerts	IP reputation check
GET	/threat-intelligence/indicators	view:alerts	List IOCs
POST	/threat-intelligence/indicators	manage:policies	Add IOC
DELETE	/threat-intelligence/indicators/{id}	manage:policies	Remove IOC

Notification Endpoints

Method	Path	Permission	Description
GET	/notifications/rules	view:alerts	Get notification rules
PUT	/notifications/rules	manage:policies	Update notification rules
POST	/notifications/test	manage:policies	Test notification channel

Testing/Simulation Endpoints

Method	Path	Auth	Description
POST	/testing/simulate	Yes	Generate simulated attack alert
GET	/testing/history	Yes	Simulation history

Prediction Endpoint

Request:

```
POST /predict
{
  "features": {
    "Flow Duration": 120.0,
    "Total Fwd Packets": 120.0,
    "Total Backward Packets": 120.0,
    "Flow Bytes/s": 120.0,
    "Flow Packets/s": 120.0,
    "Fwd Packet Length Mean": 120.0,
    "Bwd Packet Length Mean": 120.0,
    "Packet Length Std": 120.0
  }
}
```

Response:

```
{
  "prediction": "BENIGN",
  "probability": 0.97,
  "threat_score": 0.03,
  "processing_time_ms": 0.45,
  "model_version": "rf-v1.0"
}
```

Database Schema

14 tables are defined in `backend/database/models.py` :

users

Column	Type	Constraints
id	UUID	PK, default uuid4
username	VARCHAR(100)	UNIQUE, NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
hashed_password	VARCHAR(255)	NOT NULL
role	VARCHAR(20)	NOT NULL, default 'viewer'
full_name	VARCHAR(255)	nullable
is_active	BOOLEAN	default true
is_locked	BOOLEAN	default false
failed_login_attempts	INTEGER	default 0

last_login	TIMESTAMP	nullable
password_changed_at	TIMESTAMP	nullable
created_at	TIMESTAMP	default now()
updated_at	TIMESTAMP	on update now()

user_sessions

Column	Type	Constraints
id	UUID	PK
user_id	UUID	FK → users.id
jti	VARCHAR(255)	UNIQUE, NOT NULL
token_type	VARCHAR(20)	NOT NULL (access/refresh)
is_revoked	BOOLEAN	default false
ip_address	VARCHAR(45)	nullable
user_agent	VARCHAR(500)	nullable
expires_at	TIMESTAMP	NOT NULL
created_at	TIMESTAMP	default now()

auth_audit_logs

Column	Type	Constraints
id	UUID	PK
user_id	UUID	FK → users.id, nullable
username	VARCHAR(100)	nullable
role	VARCHAR(20)	nullable
action	VARCHAR(50)	NOT NULL
target	VARCHAR(255)	nullable
status	VARCHAR(20)	NOT NULL
details	JSONB	nullable
ip_address	VARCHAR(45)	nullable
timestamp	TIMESTAMP	default now()

alerts

Column	Type	Constraints
--------	------	-------------

id	UUID	PK
attack_type	VARCHAR(100)	NOT NULL
severity	VARCHAR(20)	NOT NULL
confidence	FLOAT	default 0.0
source_ip	VARCHAR(45)	nullable
destination_ip	VARCHAR(45)	nullable
source_port	INTEGER	nullable
destination_port	INTEGER	nullable
protocol	VARCHAR(20)	nullable
flow_id	VARCHAR(255)	nullable
packet_count	INTEGER	default 0
bytes_total	BIGINT	default 0
status	VARCHAR(20)	default 'new'
description	TEXT	nullable
threat_score	FLOAT	default 0.0
model_version	VARCHAR(50)	nullable
raw_data	JSONB	nullable
created_at	TIMESTAMP	default now()
updated_at	TIMESTAMP	on update now()

flow_history

Column	Type	Constraints
id	UUID	PK
flow_id	VARCHAR(255)	nullable
source_ip	VARCHAR(45)	NOT NULL
destination_ip	VARCHAR(45)	NOT NULL
source_port	INTEGER	nullable
destination_port	INTEGER	nullable
protocol	VARCHAR(20)	nullable
duration_sec	FLOAT	default 0.0

packet_count	INTEGER	default 0
byte_count	BIGINT	default 0
attack_type	VARCHAR(100)	nullable
severity	VARCHAR(20)	nullable
confidence	FLOAT	nullable
start_time	TIMESTAMP	nullable
end_time	TIMESTAMP	nullable
detected_at	TIMESTAMP	default now()

blocked_ips

Column	Type	Constraints
id	UUID	PK
ip_address	VARCHAR(45)	NOT NULL
reason	TEXT	nullable
blocked_by	VARCHAR(100)	nullable
action	VARCHAR(50)	nullable
is_active	BOOLEAN	default true
created_at	TIMESTAMP	default now()
unblocked_at	TIMESTAMP	nullable

incidents

Column	Type	Constraints
id	UUID	PK
title	VARCHAR(255)	NOT NULL
description	TEXT	nullable
severity	VARCHAR(20)	NOT NULL
priority	VARCHAR(20)	nullable
status	VARCHAR(20)	default 'open'
owner_id	UUID	FK → users.id, nullable
assigned_to	UUID	FK → users.id, nullable
alert_ids	ARRAY(UUID)	nullable

created_at	TIMESTAMP	default now()
updated_at	TIMESTAMP	on update now()
closed_at	TIMESTAMP	nullable

incident_notes

Column	Type	Constraints
id	UUID	PK
incident_id	UUID	FK → incidents.id
user_id	UUID	FK → users.id, nullable
content	TEXT	NOT NULL
created_at	TIMESTAMP	default now()

incident_timeline

Column	Type	Constraints
id	UUID	PK
incident_id	UUID	FK → incidents.id
action	VARCHAR(100)	NOT NULL
details	JSONB	nullable
user_id	UUID	FK → users.id, nullable
created_at	TIMESTAMP	default now()

response_history

Column	Type	Constraints
id	UUID	PK
alert_id	VARCHAR(255)	nullable
action	VARCHAR(100)	NOT NULL
status	VARCHAR(20)	NOT NULL
mode	VARCHAR(20)	NOT NULL
parameters	JSONB	nullable
result	JSONB	nullable
error_message	TEXT	nullable
rollback_of	UUID	nullable

rollback_available	BOOLEAN	default false
rollback_executed	BOOLEAN	default false
execution_time_ms	FLOAT	nullable
operator	VARCHAR(100)	nullable
details	JSONB	nullable
created_at	TIMESTAMP	default now()

reports

Column	Type	Constraints
id	UUID	PK
title	VARCHAR(255)	nullable
report_type	VARCHAR(50)	NOT NULL
format	VARCHAR(10)	NOT NULL
parameters	JSONB	nullable
file_path	VARCHAR(500)	nullable
status	VARCHAR(20)	default 'pending'
created_by	UUID	FK → users.id, nullable
created_at	TIMESTAMP	default now()
completed_at	TIMESTAMP	nullable

threat_indicator_cache

Column	Type	Constraints
id	UUID	PK
indicator_type	VARCHAR(50)	NOT NULL
value	VARCHAR(255)	NOT NULL
source	VARCHAR(100)	nullable
reputation	VARCHAR(20)	nullable
tags	ARRAY(TEXT)	nullable
confidence	FLOAT	nullable
threat_score	FLOAT	nullable
metadata	JSONB	nullable

last_checked	TIMESTAMP	nullable
expires_at	TIMESTAMP	NOT NULL
created_at	TIMESTAMP	default now()

notification_rules

Column	Type	Constraints
id	UUID	PK
severity	VARCHAR(20)	NOT NULL
channels	JSONB	NOT NULL
enabled	BOOLEAN	default true
created_at	TIMESTAMP	default now()
updated_at	TIMESTAMP	on update now()

prediction_logs

Column	Type	Constraints
id	UUID	PK
features	JSONB	NOT NULL
prediction	VARCHAR(100)	NOT NULL
probability	FLOAT	nullable
threat_score	FLOAT	nullable
processing_time_ms	FLOAT	nullable
model_version	VARCHAR(50)	nullable
created_at	TIMESTAMP	default now()

system_metrics

Column	Type	Constraints
id	UUID	PK
cpu_usage_percent	FLOAT	nullable
memory_usage_percent	FLOAT	nullable
timestamp	TIMESTAMP	default now()

audit_logs

Column	Type	Constraints
id	UUID	PK
user_id	UUID	nullable
action	VARCHAR(100)	NOT NULL
resource	VARCHAR(100)	nullable
resource_id	VARCHAR(100)	nullable
details	JSONB	nullable
timestamp	TIMESTAMP	default now()

Services Layer (backend/services/)

Service	File	Role
prediction_service.py	Business logic for ML prediction; validates input features, calls Predictor, logs prediction	
alert_service.py	Alert CRUD operations; interfaces with alert repository	
history_service.py	Flow history management; interfaces with flow repository	
capture_service.py	Capture lifecycle: start/stop/status; manages CaptureManager instance	
statistics_service.py	Computes threat statistics from alert data	
testing_service.py	Attack simulation: generates synthetic alerts with realistic feature values	

Frontend Architecture

Pages (22 total)

Page	Route	Description
Dashboard	/	Live stats, threat timeline, severity pie, top attacks, system health, capture controls, live packet stream
Alerts	/alerts	Paginated alert list with filters (type, severity, status, date range, IP)
Flows	/flows	Network flow table with filters
Statistics	/statistics	Threat statistics with charts
Analytics	/analytics	Analytics overview, trends, forecast
Incidents	/incidents	Incident management (create, list, detail, notes)

Response Center	/responses	Execute and manage response actions
Response History	/responses/history	Historical response records
Attack Simulation	/attack-simulation	Generate synthetic attacks for testing
Reports	/reports	Generate and download reports
Threat Intel	/threat-intel	IP reputation, IOC management
Policy Manager	/policies	Edit auto-response rules
Models	/models	View/switch ML model versions
Model Code	/model-code	View model training/pipeline code with syntax highlighting
System Health	/health	CPU, memory, disk, network, capture/model status
Notifications	/notification-settings	Edit notification routing rules
Settings	/settings	System settings
Login	/login	Authentication form
Register	/register	User registration form
Profile	/profile	User profile management
User Management	/users	Admin user CRUD
Audit Log	/audit	Auth audit log viewer with CSV export

Component Tree

```
App
├─ QueryClientProvider (TanStack React Query)
│   └─ ThemeProvider (dark/light mode)
│       └─ AlertProvider (WebSocket live alerts)
│           └─ PacketStreamProvider (WebSocket live packets)
│               └─ AuthProvider (auth state, permissions)
│                   └─ BrowserRouter
│                       ├── /login → LoginPage
│                       ├── /register → RegisterPage
│                       └─ Layout (authenticated routes)
│                           ├── Sidebar (nav links + user info)
│                           ├── TopNav (theme toggle)
│                           └─ <Outlet> → ProtectedRoute → PageComponent
```

State Management

- **Server state:** TanStack React Query (5.101) — all API data fetching, caching, refetching (10s intervals for stats/health, 5s staleTime default)
- **Auth state:** React Context (`AuthContext`) — user, tokens, permissions, login/register/logout/updateProfile/changePassword
- **Live alerts:** React Context (`AlertContext`) — rolling buffer of 200 WebSocket alerts
- **Live packets:** React Context (`PacketStreamContext`) — rolling buffer of 200 WebSocket packets, connection status
- **Theme:** React Context (`ThemeContext`) — dark/light mode, persisted to localStorage
- **Client-side routing:** React Router DOM v7 — 22 routes, ProtectedRoute wrapper with permission checks

Axios API Client (`services/api.ts`)

- Base URL from `VITE_API_BASE` env var (defaults to `/api`)
- Request interceptor: attaches `Bearer` token from localStorage
- Response interceptor: transparent 401 handling with token refresh (queues concurrent requests during refresh)
- Auto-redirect to `/login` if refresh fails

WebSocket Client (`services/websocketService.ts`)

- Singleton class-based client
- Auto-reconnect with exponential backoff (3s → 15s max)
- Event subscription model: `wsService.on( 'alert', handler )` returns unsubscribe function
- Heartbeat ping every 15s

Configuration

Backend Configuration (`backend/core/config.py`)

Uses `pydantic-settings` (Pydantic v2) for type-safe configuration from environment variables / `.env` file.

Setting	Env Variable	Default	Description
ENVIRONMENT	<code>ENVIRONMENT</code>	development	Runtime environment
API_VERSION	<code>API_VERSION</code>	v1	API version string
API_TITLE	<code>API_TITLE</code>	SOC Platform	API title
HOST	<code>HOST</code>	0.0.0.0	Bind address
PORT	<code>PORT</code>	8000	Listen port
LOG_LEVEL	<code>LOG_LEVEL</code>	INFO	Logging level

DB_HOST	DB_HOST	localhost	PostgreSQL host
DB_PORT	DB_PORT	5432	PostgreSQL port
DB_USER	DB_USER	postgres	DB username
DB_PASSWORD	DB_PASSWORD	lenovo	DB password
DB_NAME	DB_NAME	ids_soc	Database name
CORS_ORIGINS	CORS_ORIGINS	["*"]	Allowed CORS origins
JWT_SECRET_KEY	JWT_SECRET_KEY	change-this-to-a-random-secret-key	JWT signing key
JWT_ACCESS_TOKEN_EXPIRE_MINUTES	JWT_ACCESS_TOKEN_EXPIRE_MINUTES	60	Access token TTL
JWT_REFRESH_TOKEN_EXPIRE_DAYS	JWT_REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh token TTL
SMTP_HOST	SMTP_HOST	smtp.example.com	SMTP server
SMTP_PORT	SMTP_PORT	587	SMTP port
SMTP_USER	SMTP_USER	notifications@example.com	SMTP username
SMTP_PASSWORD	SMTP_PASSWORD	your-smtp-password	SMTP password
TELEGRAM_BOT_TOKEN	TELEGRAM_BOT_TOKEN	(empty)	Telegram bot token
TELEGRAM_CHAT_ID	TELEGRAM_CHAT_ID	(empty)	Telegram chat ID
ALIENVAULT_OTX_API_KEY	ALIENVAULT_OTX_API_KEY	(empty)	OTX API key
VIRUSTOTAL_API_KEY	VIRUSTOTAL_API_KEY	(empty)	VirusTotal API key
REDIS_HOST	REDIS_HOST	localhost	Redis host
REDIS_PORT	REDIS_PORT	6379	Redis port
REDIS_PASSWORD	REDIS_PASSWORD	(empty)	Redis password

PROMETHEUS_PORT	PROMETHEUS_PORT	9090	Prometheus port
GRAFANA_PORT	GRAFANA_PORT	3000	Grafana port
DB_POOL_SIZE	DB_POOL_SIZE	10	Connection pool size
DB_MAX_OVERFLOW	DB_MAX_OVERFLOW	20	Max overflow connections

Frontend Configuration

Variable	Default	Description
VITE_API_BASE	/api	Backend API base URL
VITE_WS_URL	ws://{hostname}:8000/ws/alerts	WebSocket URL

Middleware Stack (backend/core/middleware.py)

Middleware	Order	Purpose
RequestIDMiddleware	1st	Adds unique X-Request-ID header to every request/response
RequestLoggingMiddleware	2nd	Logs method, path, status, duration, request ID for every request
RateLimitMiddleware	3rd	Sliding window rate limiter per IP (configurable max requests / window)

All three are ASGI middleware classes added in `main.py` via `app.add_middleware()` .

Error Handling

- FastAPI's built-in validation (422) for request body/query validation errors
- Auth module raises typed exceptions (e.g., `InvalidCredentialsError` , `PermissionDeniedError`) caught by FastAPI exception handlers
- `RateLimitMiddleware` returns 429 when limit exceeded
- Response engine records errors in `response_history.error_message`
- Frontend error boundaries: `ErrorDisplay` component with retry button
- All API errors return consistent JSON: `{"error": "...", "detail": "..."}`

Security

- **Authentication:** JWT-based with access (60min) + refresh (7 days) tokens
- **Password hashing:** bcrypt via passlib
- **Authorization:** Role-based (3 roles) with 15 granular permissions

- **Rate limiting:** Nginx level (100 r/s API, 20 r/s WS) + application level (sliding window)
- **Security headers:** nginx adds X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy, CSP, HSTS
- **TLS:** nginx terminates HTTPS with configurable certificates
- **Sensitive path blocking:** nginx denies access to `.env`, `.git`, `node_modules`, `venv`
- **Session management:** Refresh tokens stored in DB with revocation support
- **CORS:** Configurable origins (defaults to wildcard in development)
- **Production considerations:** JWT secret must be changed from default; DB password must be strong random value; CORS should be restricted to specific domains

Deployment

Docker Compose Services

Service	Image/Base	Port	Profiles
postgres	postgres:16-alpine	5432	always
redis	redis:7-alpine	6379	with-cache
backend	python:3.14-slim (custom)	8000	always
frontend	nginx:1.27-alpine (custom)	80, 443	always
prometheus	prom/prometheus (custom)	9090	always
grafana	grafana/grafana (custom)	3001	always
elasticsearch	elasticsearch:8.12.0	9200	with-elk
logstash	logstash:8.12.0	5000, 9600	with-elk
kibana	kibana:8.12.0	5601	with-elk
filebeat	filebeat:8.12.0	-	with-elk

Dockerfiles

- **backend.Dockerfile:** Multi-stage build (builder for pip install, slim runtime); installs `libpcap-dev`; copies backend source + models + `.env`; healthcheck on `/health`
- **frontend.Dockerfile:** Multi-stage (node:22-alpine builder, nginx:1.27-alpine final); serves built static files; custom `nginx.conf`
- **nginx.Dockerfile:** Self-generates self-signed SSL cert; copies `nginx.conf`
- **prometheus.Dockerfile:** Custom config + alert rules
- **grafana.Dockerfile:** Provisioned datasources (Prometheus + Elasticsearch) + SOC dashboard

Scaling

- Backend supports horizontal scaling via Docker Compose `replicas`
- Database connection pool: `pool_size=10`, `max_overflow=20`
- Unicorn workers: $(2 * \text{CPU_CORES}) + 1$ (default 4 in Dockerfile)

- Redis optional caching profile

Monitoring

- **Prometheus:** Scrapes metrics from backend (`/metrics`), PostgreSQL exporter, and node-exporter
- **Alert rules:** BackendDown, HighThreatRate, HighPredictionLatency, DatabaseConnectionHigh, HighCPUUsage
- **Grafana:** Pre-configured Prometheus + Elasticsearch datasources; SOC overview dashboard (`soc_overview.json`)

Logging (ELK Stack)

- **Filebeat:** Ships Docker container logs to Logstash (auto-discovers backend containers)
- **Logstash:** Parses JSON backend logs (ISO8601 timestamps) and nginx logs (combined Apache format)
- **Elasticsearch:** Single-node, security disabled; indices: `soc-logs-YYYY.MM.DD`
- **Kibana:** Connected to Elasticsearch for log visualization

Request Lifecycle (Full Trace)

Example: User views Alerts page

1. **Browser** navigates to `/alerts`
2. **React Router** renders `<Alerts />` inside `<Layout />`
3. **Alerts** component calls `useQuery(['alerts', filters], () => fetchAlerts(filters))`
4. **fetchAlerts** makes `GET /alerts?page=1&page_size=50` via axios
5. **nginx** terminates TLS, applies rate limit (100 r/s), proxies to `http://backend:8000/alerts`
6. **RequestIDMiddleware** adds `X-Request-ID` header
7. **RequestLoggingMiddleware** logs `GET /alerts -> 200 45ms req-abc123`
8. **RateLimitMiddleware** checks sliding window for client IP
9. **FastAPI** routes to `get_alerts()` handler in `routes/alerts.py`
10. **Auth dependency** validates JWT from `Authorization: Bearer <token>` header
11. **Authorization** checks `view:alerts` permission
12. **Route handler** calls `alert_service.get_alerts(skip, limit, filters)`
13. **Service** calls `AlertRepository.find_all(skip, limit, filters)`
14. **Repository** builds SQLAlchemy async query, executes via `AsyncSession`
15. **PostgreSQL** returns matching rows
16. **Response** serialized as JSON paginated response
17. **nginx** returns response to browser
18. **React Query** caches result (staleTime: 5s), renders alert table
19. **Simultaneously:** `AlertContext` receives live alerts via WebSocket (`ws://host:8000/ws/alerts`), displayed as live count

Known Limitations

1. **Notification services are stubs:** Email, Telegram, and Webhook notification channels only log to console. No real SMTP/Telegram/HTTP integration.

2. **Threat intelligence feeds are stubs:** All external feed integrations return empty lists. No real API calls to AlienVault OTX or VirusTotal.
 3. **Response engine simulation-only actions:** `kill_process`, `isolate_host`, `quarantine_host`, `restart_service`, `revoke_session`, `vpn_isolation` — these are simulation only (log only, no real system impact).
 4. **Alembic migrations not configured:** `backend/database/migrations/` directory is empty. Tables are created via `Base.metadata.create_all()` on startup, which is not suitable for production schema migrations.
 5. **JWT secret is hardcoded:** `backend/core/config.py` defaults to `change-this-to-a-random-secret-key`. This must be set via `JWT_SECRET_KEY` environment variable in production.
 6. **No real Redis integration:** Redis service is defined in docker-compose but not actually used in the backend code (no caching or session storage).
 7. **No persistent model version tracking:** The active model version is stored in-memory only (Python variable), not persisted to database. Restart reverts to default version.
 8. **Database connection pooling:** configured but uses default `pool_size/max_overflow` without connection monitoring.
 9. **Frontend tests:** Not present. Only backend tests exist (25 test files).
 10. **Empty component directories:** `components/alerts/`, `components/tables/`, `components/statistics/` are empty — page-level components use inline rendering instead of reusable table/statistics components.
-

Conclusion

The AI-Powered SOC Platform is a comprehensive, well-architected IDS/SOC system with clear separation of concerns: presentation (React + TypeScript), API (FastAPI), business logic (services), data access (repositories), and infrastructure (deployment). The ML pipeline is fully functional with versioned model artifacts, the detection pipeline integrates capture → flow → predict → alert in real-time, and the response engine provides automated countermeasures with rollback support.

Key strengths: modular design, thorough test coverage (229 tests), comprehensive API surface, production-grade deployment configuration (Docker, nginx, Prometheus/Grafana, ELK stack), and detailed documentation.

Main areas needing production hardening: notification and threat intelligence integration, Alembic migrations, JWT secret management, persistent model version tracking, and Redis integration.